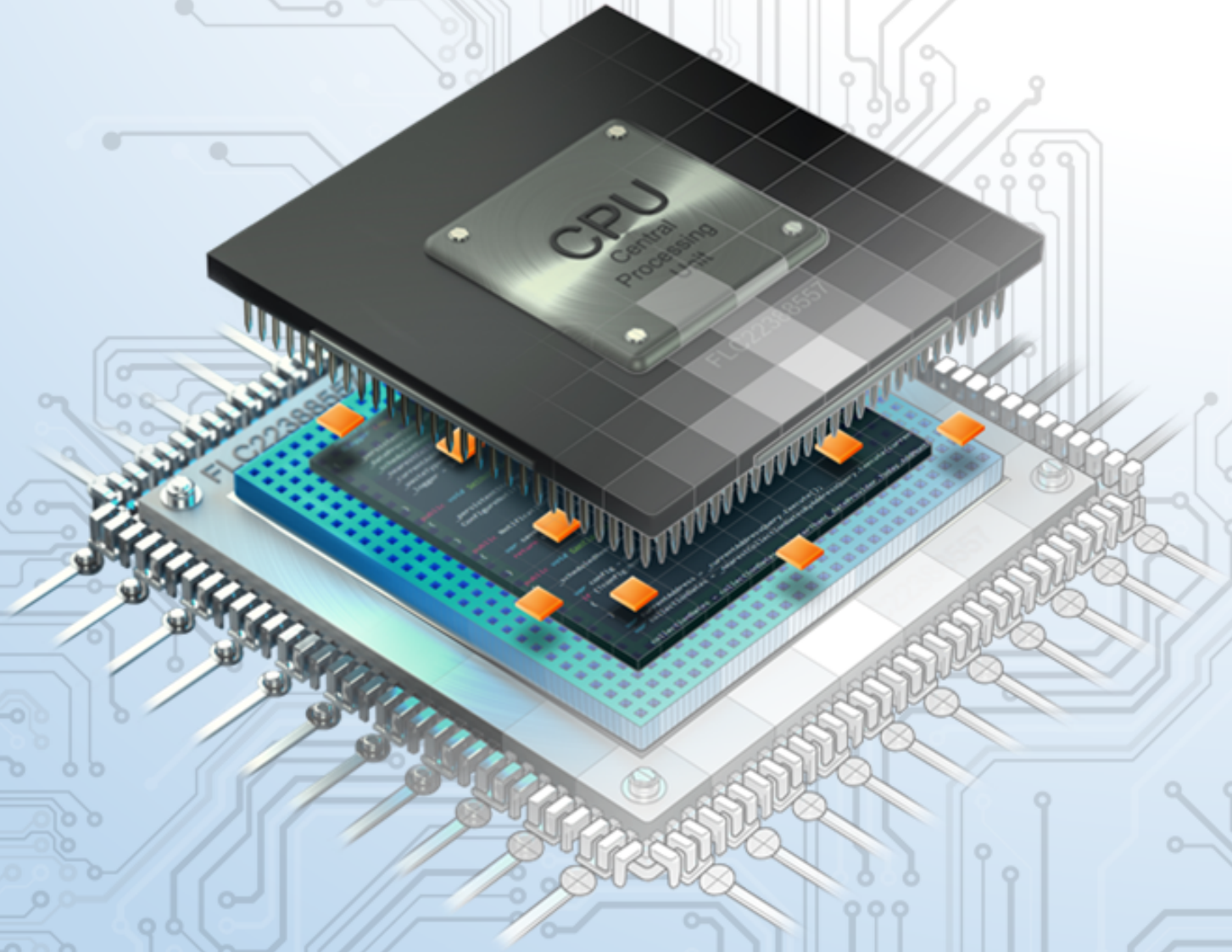
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
COMPUTER ENGINEERING

Microcontroller

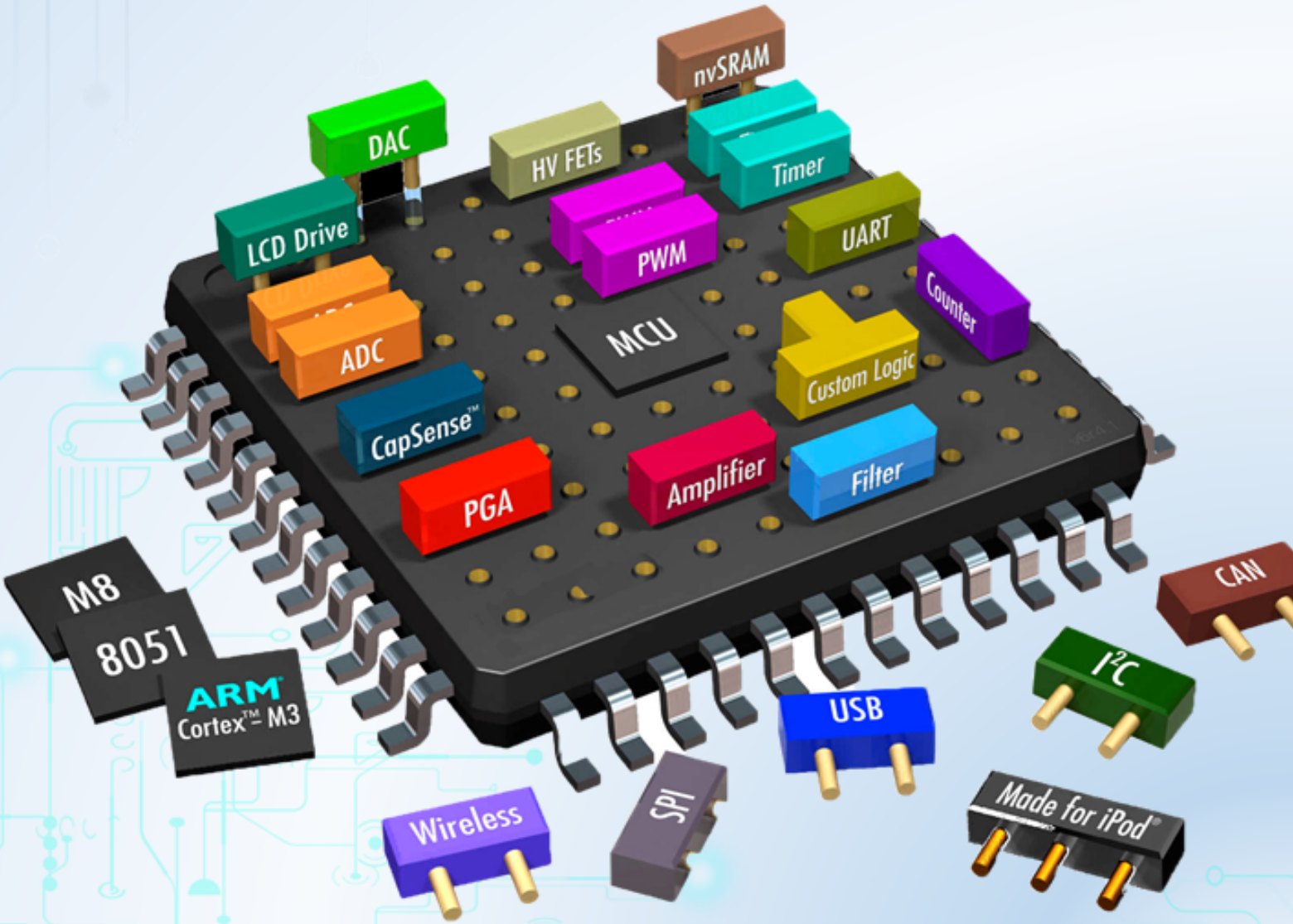


Lecture: Nguyễn Thiên Ân
Student: Đỗ Huy Minh Dũng

Mục lục

CHƯƠNG 1

LED Animations



1 Introduction

In this manual, the STM32CubeIDE is used as an editor to program the ARM microcontroller. STM32CubeIDE is an advanced C/C++ development platform with peripheral configuration, code generation, code compilation, and debug features for STM32 microcontrollers and microprocessors.

Hình 1.1: *STM32Cube IDE for STM32 Programming*

The most interest of STM32CubeIDE is that after the selection of an empty STM32 MCU or MPU, or preconfigured microcontroller or microprocessor from the selection of a board, the initialization code generated automatically. At any time during the development, the user can return to the initialization and configuration of the peripherals or middleware and regenerate the initialization code with no impact on the user code. This feature can simplify the initialization process and speedup the development application running on STM32 micro-controller. The software can be downloaded from the link bellow:

https://ubc.sgp1.digitaloceanspaces.com/BKU_Softwares/STM32/stm32cubeide_1.7.0.zip

Moreover, for a hangout class, the program is firstly simulated on Proteus. Students are also supposed to download and install this software as well:

https://ubc.sgp1.digitaloceanspaces.com/BKU_Softwares/STM32/Proteus_8.10_SP0_Pro.exe

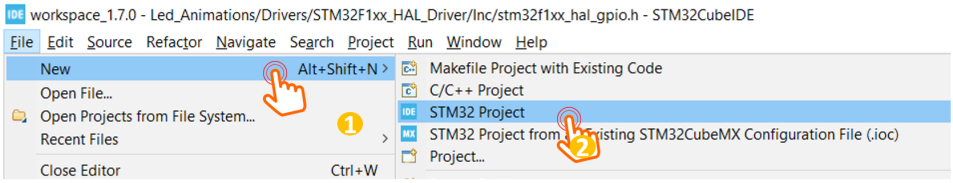
The rest of this manual consists of:

- Create a project on STM32Cube IDE
- Create a project on Proteus
- Simulate the project on Proteus

Finally, students are supposed to finish 10 different projects.

2 First project on STM32Cube

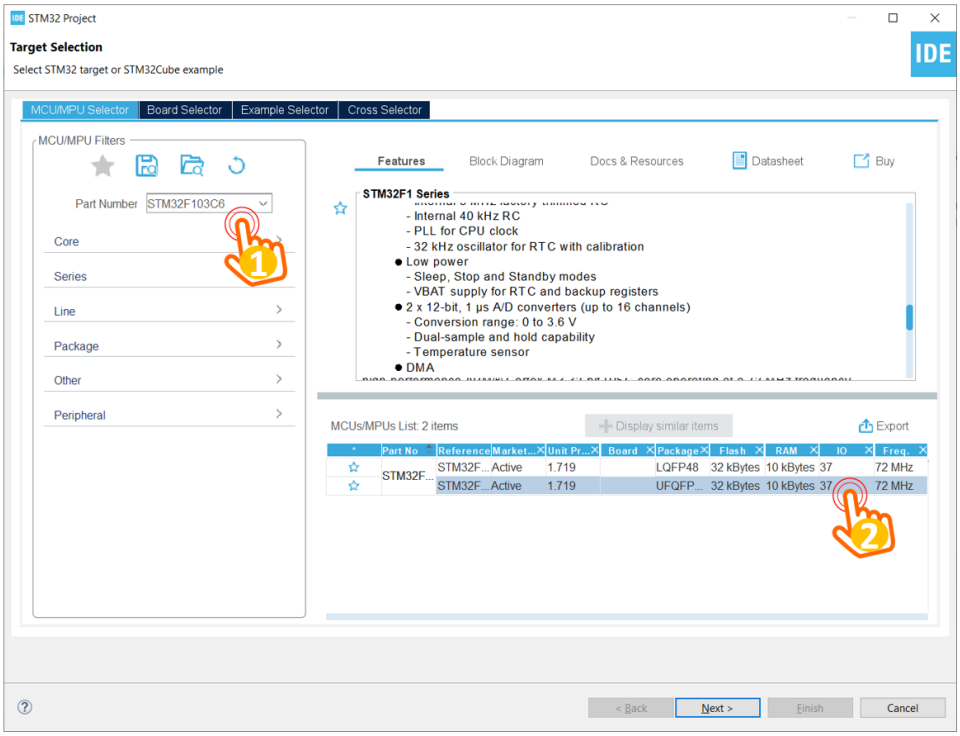
Step 1: Launch STM32CubeIDE, from the menu **File**, select **New**, then chose **STM32 Project**



Hình 1.2: Create a new project on STM32CubeIDE

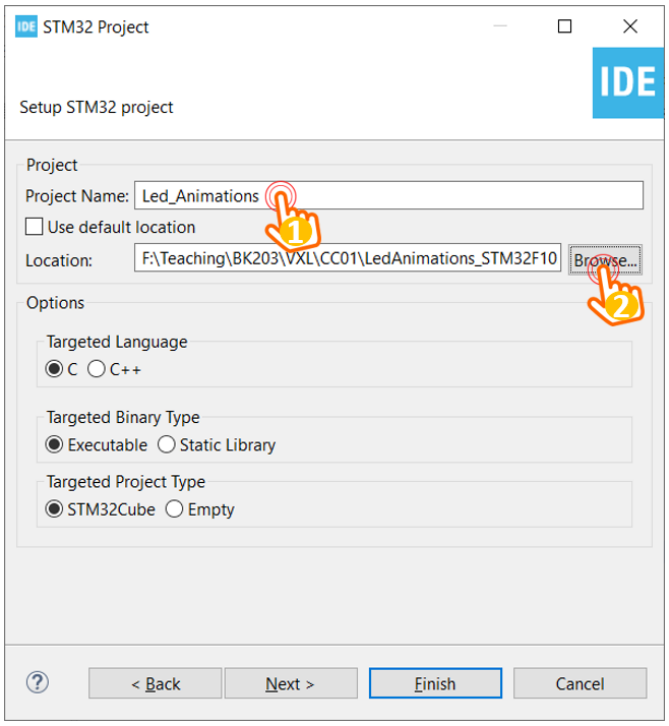
The IDE needs to download some packages, which normally takes time in this first time a project is created.

Step 2: Select the STM32F103C6 in the following dialog, then click on **Next**



Hình 1.3: Select the target device

Step 3: Provide the **Name** and the **Location** for the project.



Hình 1.4: Select the target device

It is important to notice that the **Targeted Project Type** should be **STM32Cube**. In the case this option is disable, step 1 must be repeated. The location path should not contain special characters (e.g. the space). Finally, click on the **Next** button.

Step 4: On the last dialog, just keep the default firmware version and click on **Finish** button.

Step 5: The project is created and the wizard for configuration is display. This utility from CubeIDE can simplify the configuration process for an ARM microcontroller like the STM32.

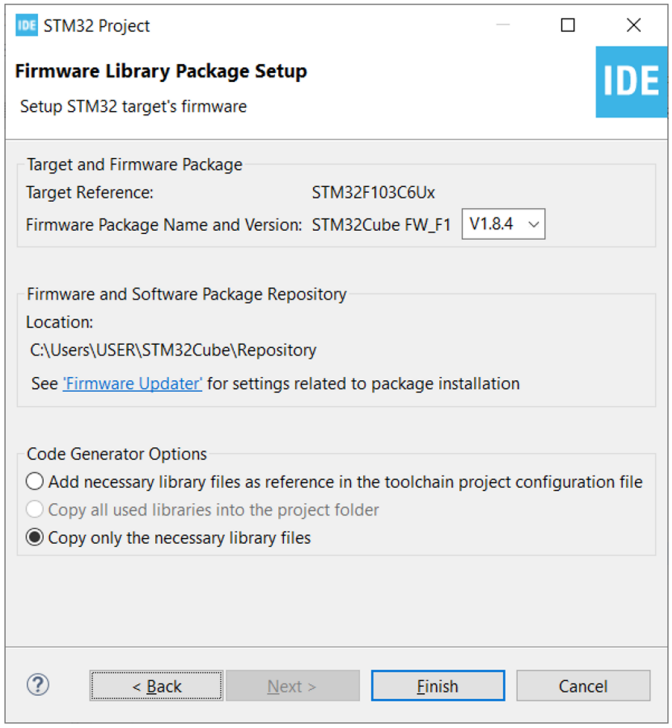
From the configuration windows, select **Pin configuration**, select the pin **PA5** and set to **GPIO Output** mode, since this pin is connected to an LED in the STM32 development kit.

Step 6: Right click on PA5 and select **Enter user lable**, and provide the name for this pin (e.g. **LED_RED**). This step helps programming afterward more memorable.

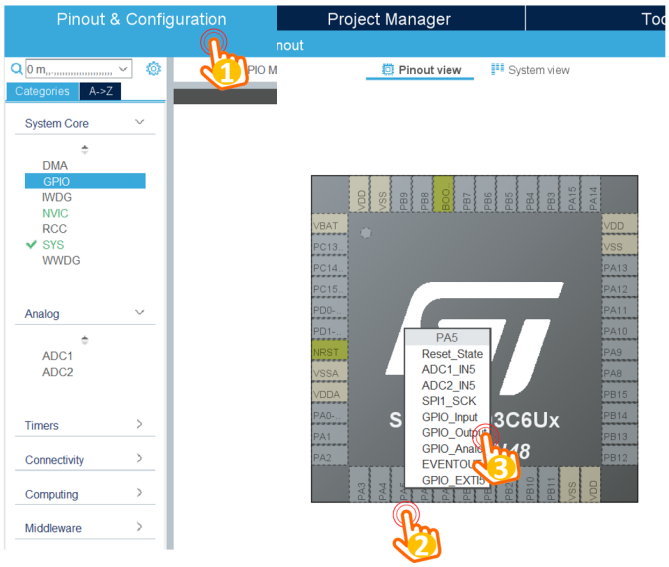
Finally, save the configuration process by pressing **Ctrl + S** and confirm this step by clicking on **OK** button. The code generation is started.

Step 7: Implement the first blinky project in the main function as follow:

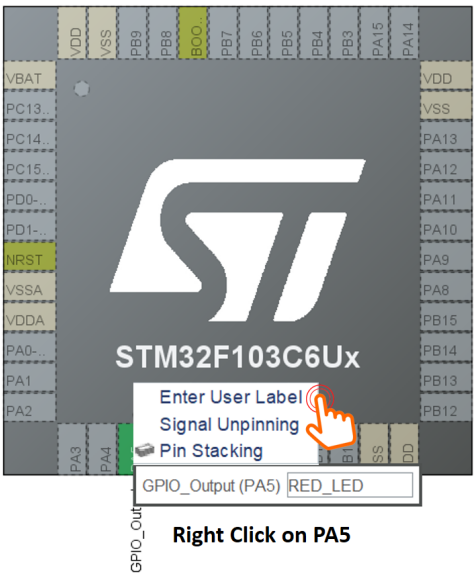
```
int main(void)
{
  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */
```



Hình 1.5: Keep default firmware version



Hình 1.6: Set PA5 to GPIO Output mode



Hình 1.7: Provide a name for PA5

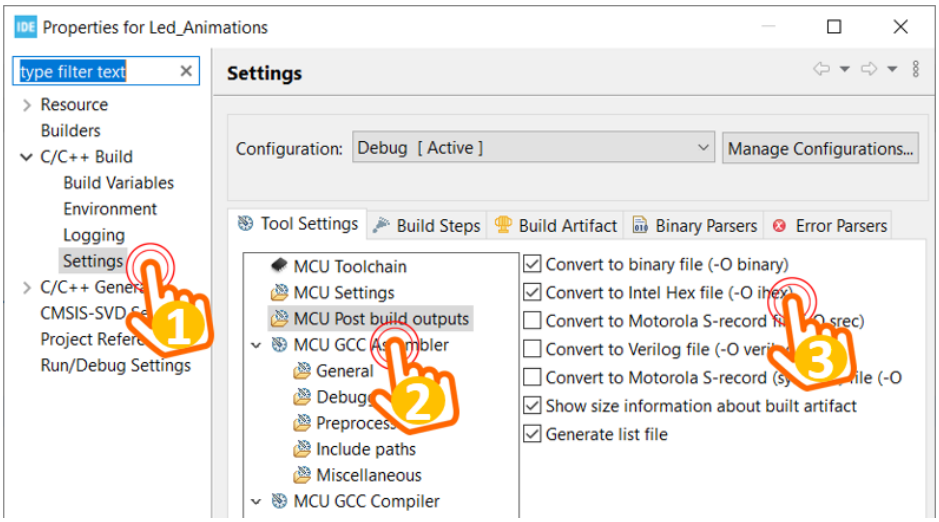
```
6
7   /* MCU Configuration
     ---------------------------------------------------------
     */
8
9   /* Reset of all peripherals, Initializes the Flash
     interface and the Systick. */
10  HAL_Init();
11
12  /* USER CODE BEGIN Init */
13
14  /* USER CODE END Init */
15
16  /* Configure the system clock */
17  SystemClock_Config();
18
19  /* USER CODE BEGIN SysInit */
20
21  /* USER CODE END SysInit */
22
23  /* Initialize all configured peripherals */
24  MX_GPIO_Init();
25  /* USER CODE BEGIN 2 */
26
27  /* USER CODE END 2 */
28
29  /* Infinite loop */
30  /* USER CODE BEGIN WHILE */
31
32  while (1)
33  {
```

```
34      HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
35      HAL_Delay(1000);
36      /* USER CODE END WHILE */
37
38      /* USER CODE BEGIN 3 */
39    }
40    /* USER CODE END 3 */
41  }
```

Program 1.1: First blinky LED project

Actually, what is added to the main function is line number 34 and 35. Please put your code in a right place, otherwise it can be deleted when the code is generated (e.g. change the configuration of the project). When coding, frequently use the suggestions by pressing **Ctrl+Space**.

Step 7: Due to the simulation on Proteus, the hex file should be generated from STM32Cube IDE. From menu **Project**, select **Properties** to open the dialog bellow:



Hình 1.8: Config for hex file output

Navigate to **C/C++ Build**, select **Settings, MCU Post build outputs**, and check to the **Intel Hex file**.

Step 8: Build the project by clicking on menu **Project** and select **Build Project**. Please check on the output console of the IDE to be sure that the hex file is generated, as follow:

```
22:36:06 **** Incremental Build of configuration Debug for project Led_Animations ****
make -j8 all
arm-none-eabi-size   Led_Animations.elf
   text    data     bss     dec     hex filename
   4596      20    1572    6188    182c Led_Animations.elf
Finished building: default.size.stdout


22:36:06 Build Finished. 0 errors, 0 warnings. (took 272ms)
```

Hình 1.9: Compile the project and generate Hex file

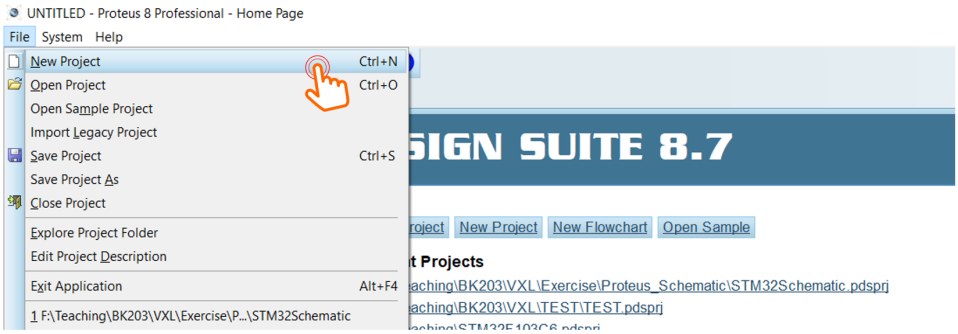
The hex file is located under the **Debug** folder of your project, which is used for the simulation in Proteus afterward. In the case a development kit is connected to your PC, from menu **Run**, select **Run** to download the program to the hardware platform.

In the case there are multiple project in a work-space, double click on the project name to activate this project. Whenever a project is built, check the output files to make sure that you are working in a right project.

3 Simulation on Proteus

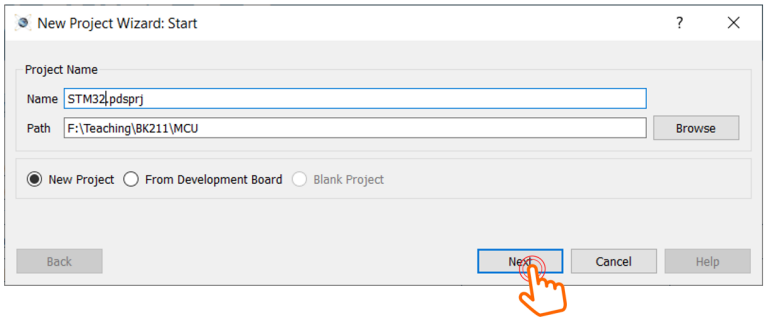
For an online training, a simulation on Proteus can be used. The details to create an STM32 project on Proteus are described bellow.

Step 1: Launch Proteus (**with administration access**) and from menu **File**, select **New Project**.



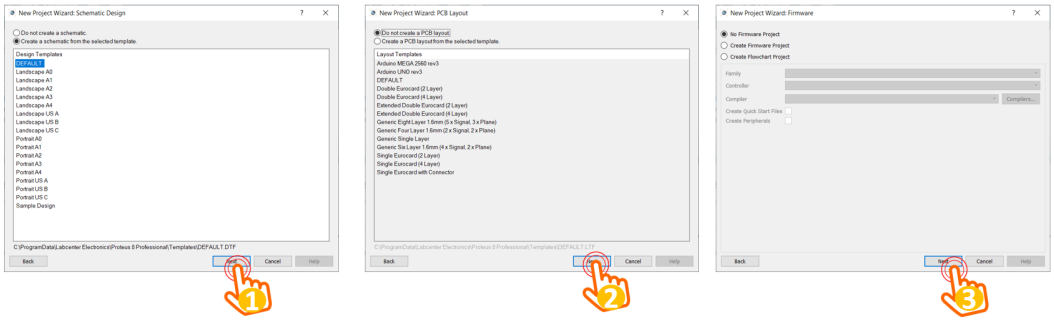
Hình 1.10: Create a new project on Proteus

Step 2: Provide the name and the location of the project, then click on **Next** button.



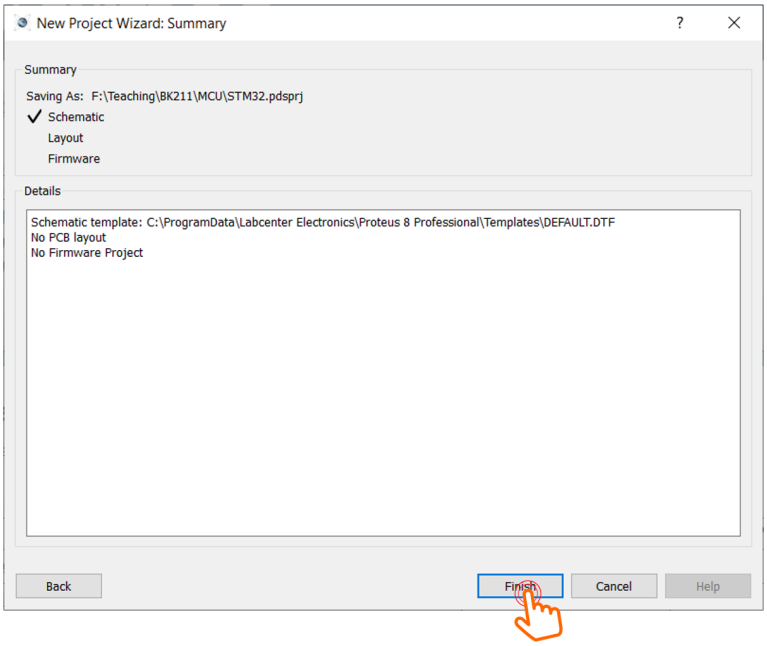
Hình 1.11: Provide project name and location

Step 3: For following dialog, just click on **Next** button as just a schematic is required for the lab.



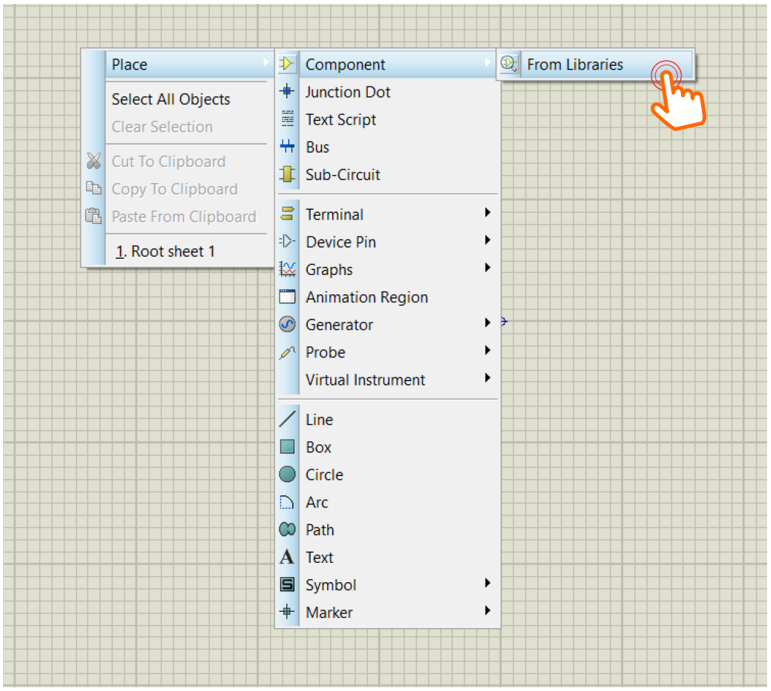
Hình 1.12: Keep the default options by clicking on Next

Step 4: Finally, click on **Finish** button to close the project wizard.



Hình 1.13: Finish the project wizard

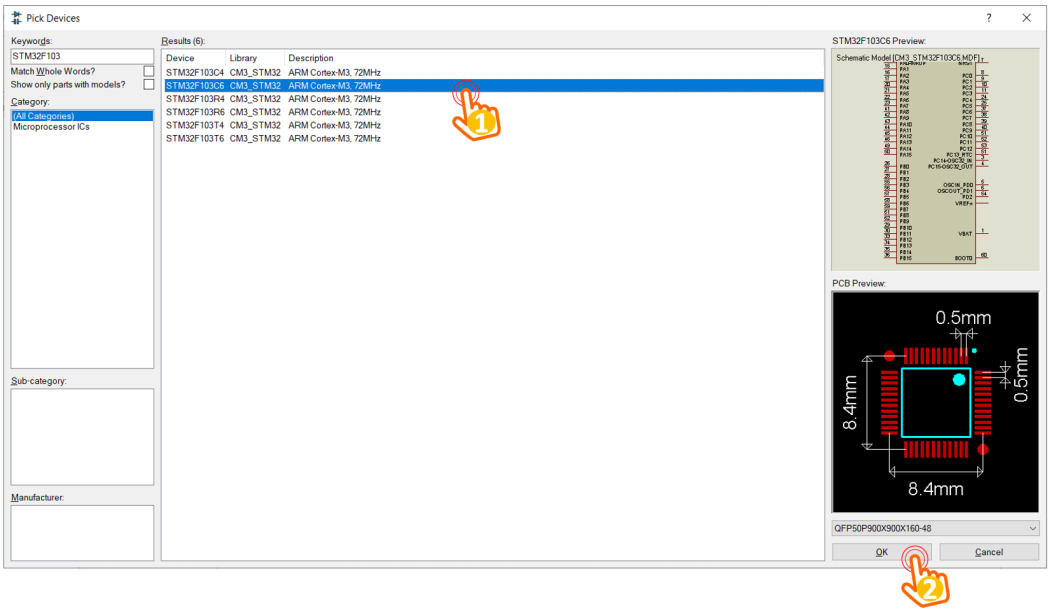
Step 5: On the main page of the project, right click to select **Place, Components, From Libraries**, as follows:



Hình 1.14: Select a component from the library

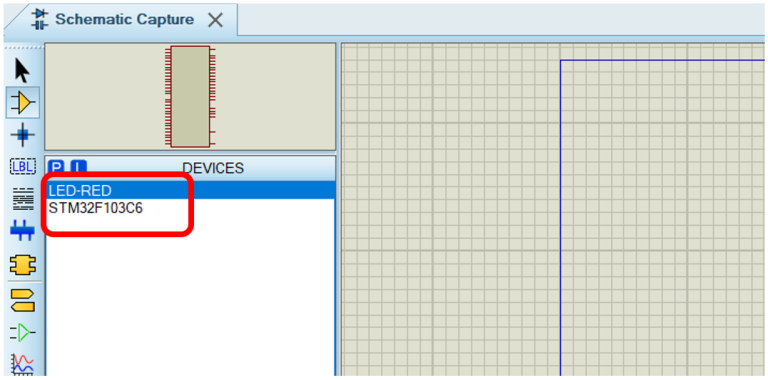
If there is an error with no library found, please restart the Proteus software with Run as administrator option.

Step 6: From the list of components in the library, select STM32F103C6, as follows:



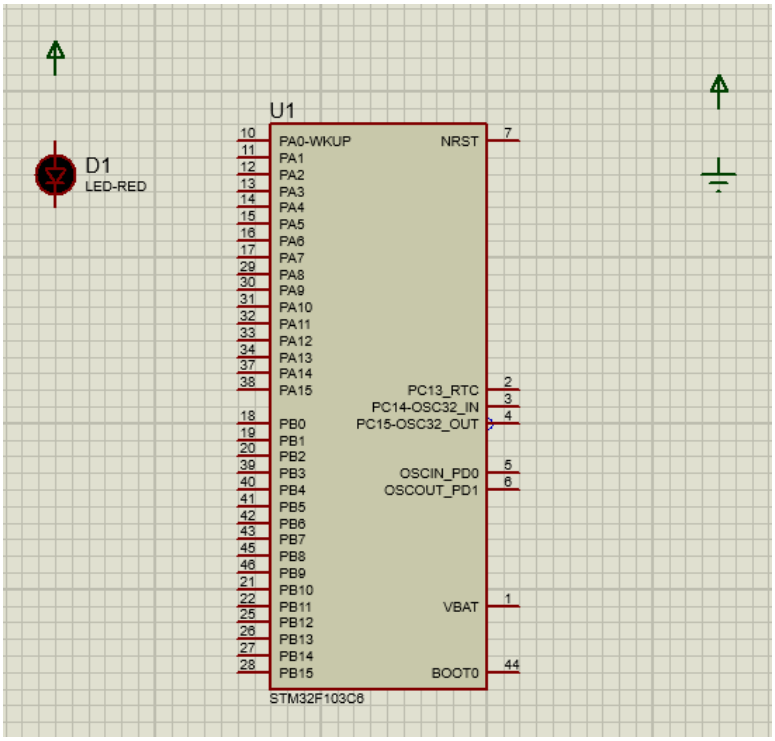
Hình 1.15: Select STM32F103C6

Repeat step 5 and 6 to select an LED, named **LED-RED** in Proteus. Finally, these components are appeared on the DEVICES windows, which is on left hand side as follows:



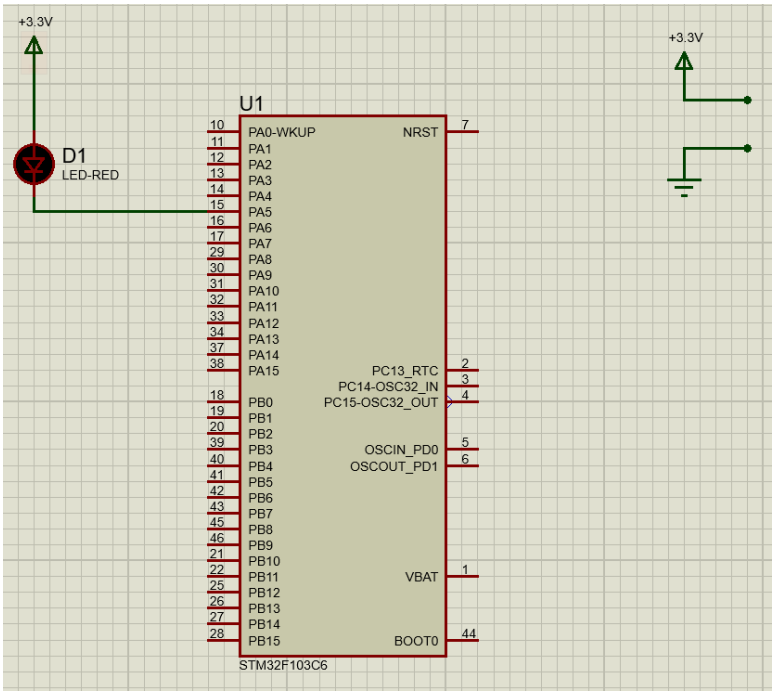
Hình 1.16: STM32 and an LED in the project

Step 7: Place the components to the project: right click on the main page, select on **Place, Component**, and select device added in Step 6. To add the Power and the Ground, right click on the main page, select on **Place, Terminal**. The result in this step is expected as follows:



Hình 1.17: *Place components to the project*

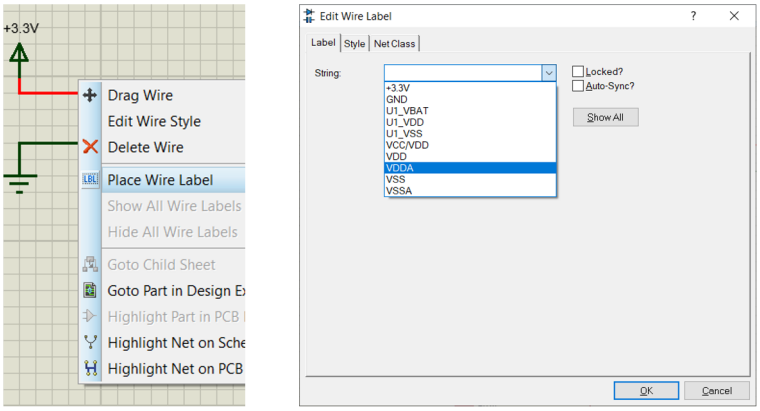
Step 8: Start wiring the circuit. The negative pin of the LED is connected to PA5 while its positive pin is connected to the power supply. For the power and the ground on the right, just make a short wire, which will labeled in the next step.



Hình 1.18: *Connect components and set the power to 3.3V*

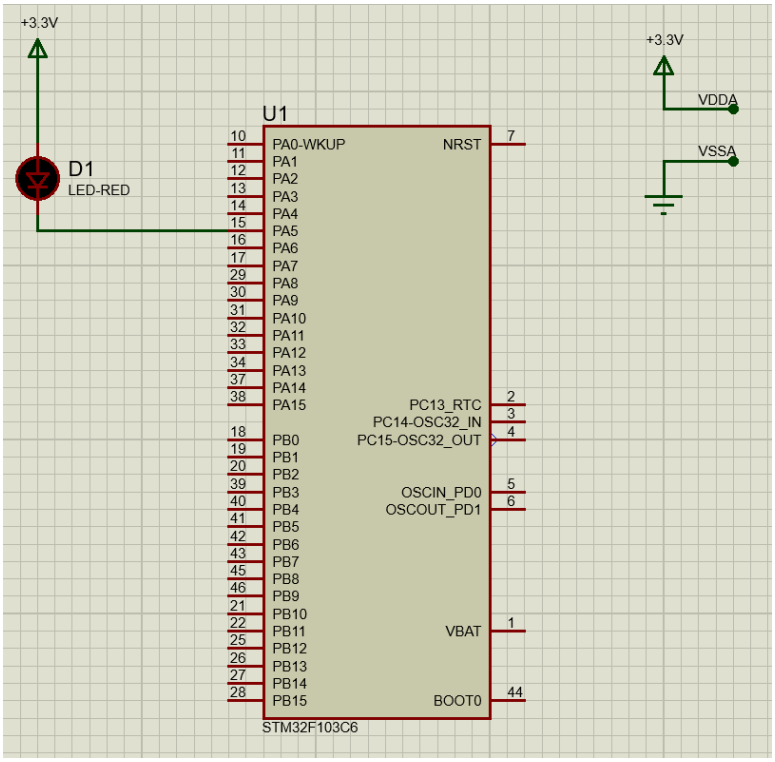
In this step, also double click on the power supply in order to provide the String property to **+3.3V**.

Step 8: Right click on the wire of the power supply and the ground, and select **Place wire Label**



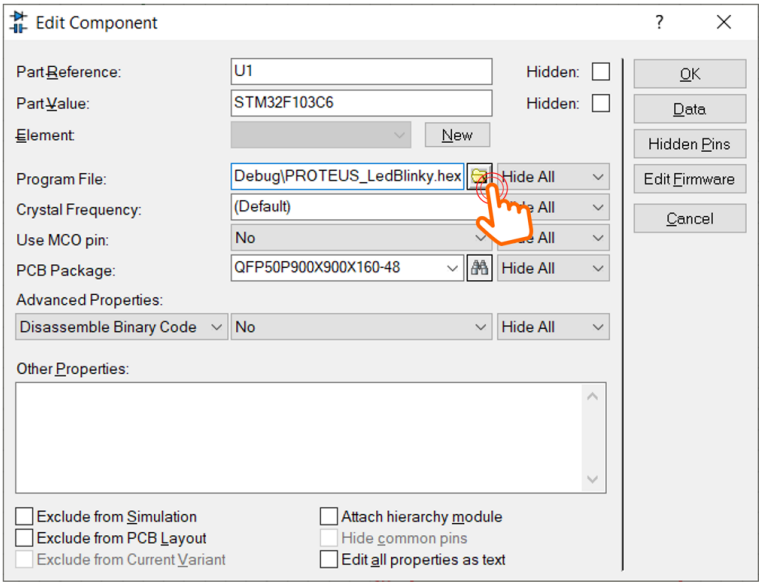
Hình 1.19: *Place label for Power and Ground*

This step is required as VDDA and VSSA of the STM32 must be connected to provide the reference voltage. Therefore, VDDA is connected to 3.3V, while the VSSA is connected to the Ground. Finally, the image of our schematic is shown bellow:



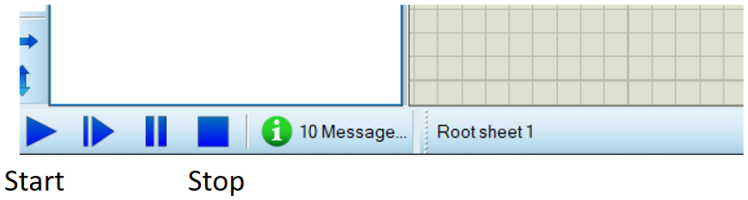
Hình 1.20: *Finalize the schematic*

Step 9: Double click on the STM32, and set the **Program File** to the Hex file, which is generated from Cube IDE, as following:



Hình 1.21: *Set the program of the STM32 to the hex file from Cube IDE*

From now, the simulation is ready to start by clicking on the menu **Debug**, and select on **Run simulation**. To stop the simulation, click on **Debug** and select **Stop VMS Debugging**. Moreover, there are some quick access bottom on the left corner of the Proteus to start or stop the simulation, as shown following:



Hình 1.22: *Quick access buttons to start and stop the simulation*

If everything is success, students can see the LED is blinking every second. Please stop the simulation before updating the project, either in Proteus or STM32Cube IDE. However, the step 9 (set the program file for STM32 in Proteus) is required to do once. Beside the toggle instruction, student can set or reset a pin as following:

```
while (1){
  HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
   GPIO_PIN_SET);
  HAL_Delay(1000);
  HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
   GPIO_PIN_RESET);
  HAL_Delay(1000);
}
```

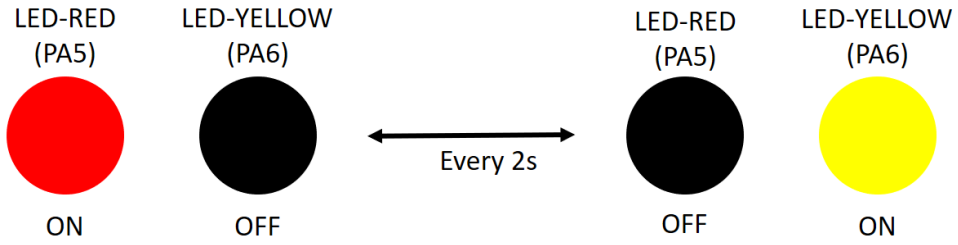
Program 1.2: An example for LED blinky

4 Exercise and Report

4.1 Exercise 1

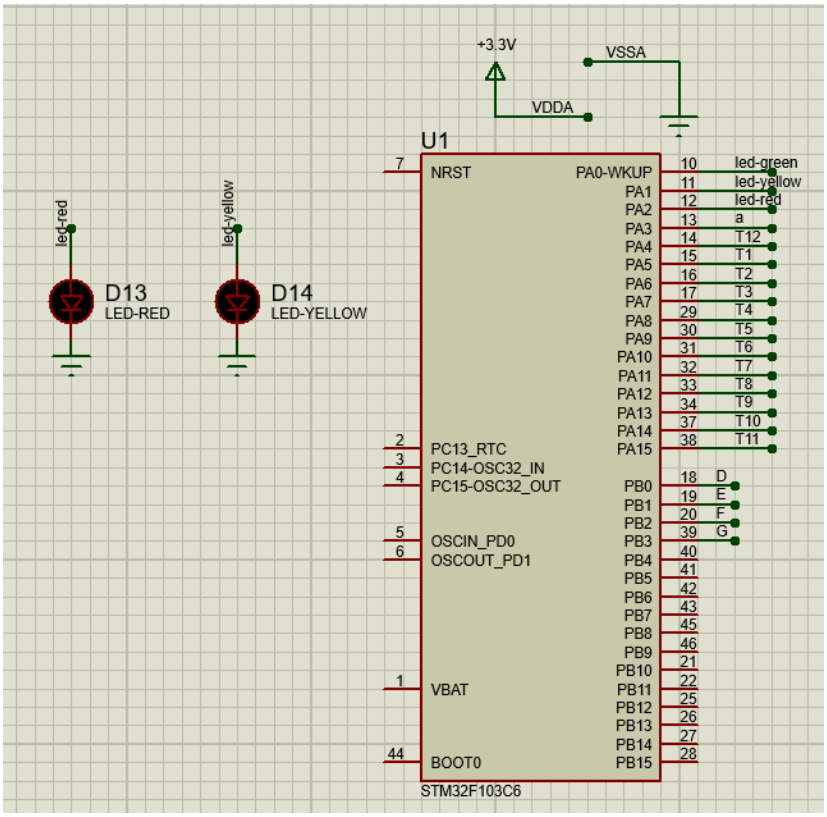
From the simulation on Proteus, one more LED is connected to pin **PA6** of the STM32 (negative pin of the LED is connected to PA6). The component suggested in this exercise is **LED-YELLOW**, which can be found from the device list.

In this exercise, the status of two LEDs are switched every 2 seconds, as demonstrated in the figure bellow.



Hình 1.23: *State transitions for 2 LEDs*

Report 1: Depict the schematic from Proteus simulation in this report. The caption of the figure is a downloadable link to the Proteus project file (e.g. a github link).



Hình 1.24: Proteus file for lab 1 here

Report 2: Present the source code in the infinite loop while of your project. If a user-defined functions is used, it is required to present in this part. A brief description can be added for this function (e.g. using comments).

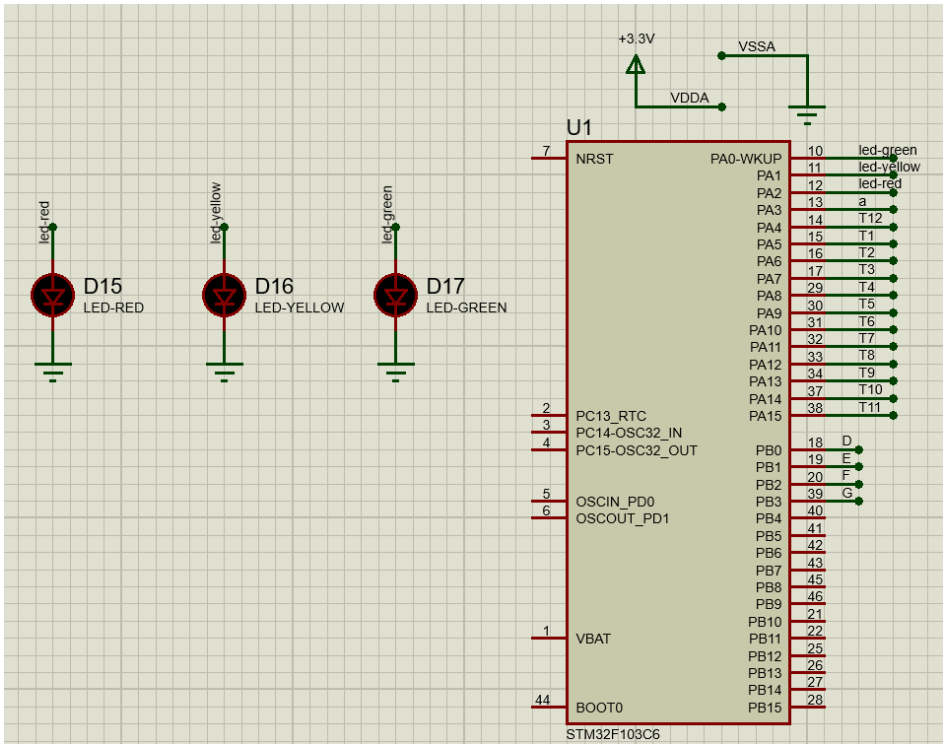
```
int state = 0;
void ex1(){
    if (state <= 1) {
      HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
   GPIO_PIN_SET);
      HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port,
   LED_YELLOW_Pin, GPIO_PIN_RESET);
    }
    else {
      HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin,
   GPIO_PIN_RESET);
      HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port,
   LED_YELLOW_Pin, GPIO_PIN_SET);
    }
    state++;
    if(state >= 4)
    {
      state = 0;
    }
    HAL_Delay (1000) ;
}
  while (1)
    {
    ex1();
    }
```

4.2 Exercise 2

Extend the first exercise to simulate the behavior of a traffic light. A third LED, named **LED-GREEN** is added to the system, which is connected to **PA7**. A cycle in this traffic light is 5 seconds for the RED, 2 seconds for the YELLOW and 3 seconds for the GREEN. The LED-GREEN is also controlled by its negative pin.

Similarly, the report in this exercise includes the schematic of your circuit and a your source code in the while loop.

Report 1: Present the schematic.



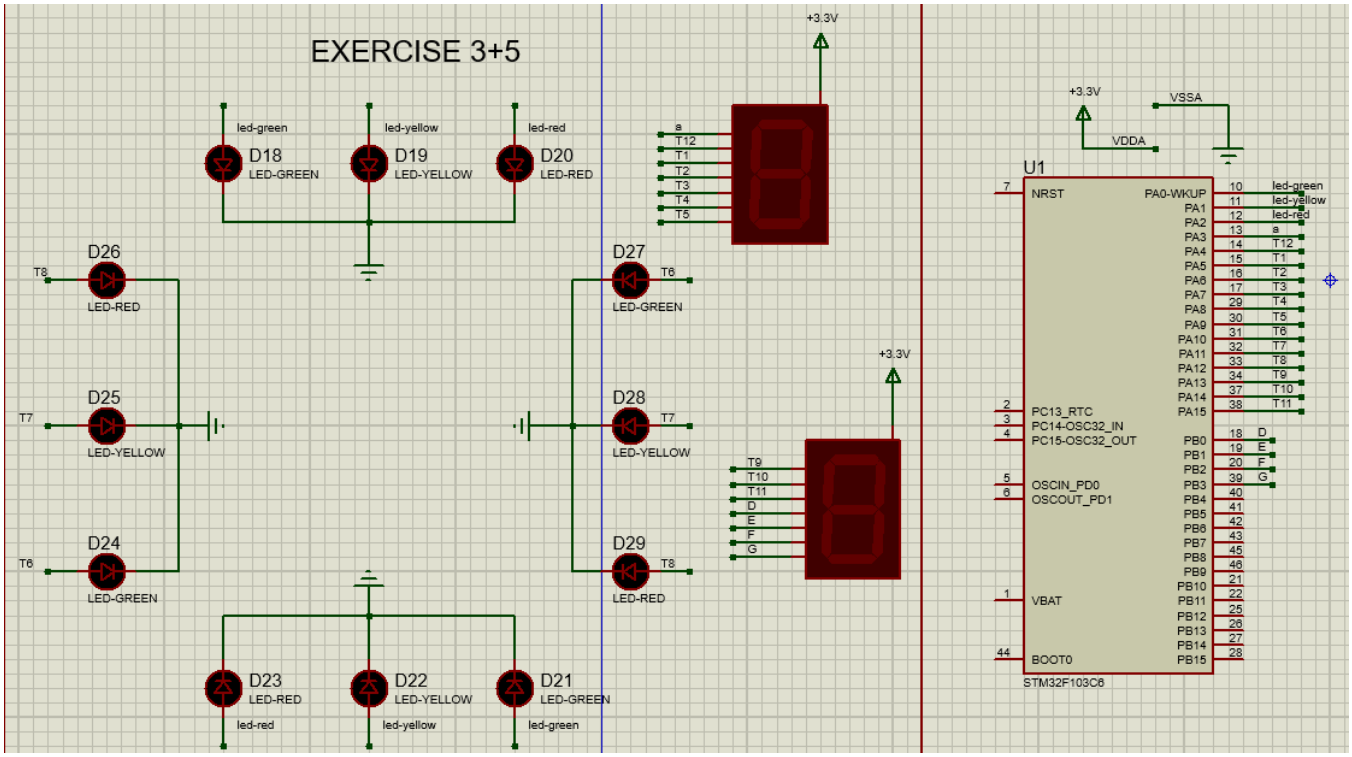
Report 2: Present the source code in while.

```
int count = 9;
void ex2(){
    if(count < 5){
      HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, 1);
      HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port,
  LED_YELLOW_Pin, 0);
      HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
   0);
    }
    else if(count < 7){
      HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, 0);
      HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port,
  LED_YELLOW_Pin, 1);
      HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
   0);
    }
    else{
      HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, 0);
      HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port,
  LED_YELLOW_Pin, 0);
      HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
   1);
    }
    count--;
    if(count < 0){
      count = 9;
    }
```

```
    HAL_Delay(1000);
}
while (1){
  ex2();
}
```

4.3 Exercise 3

Extend to the 4-way traffic light. Arrange 12 LEDs in a nice shape to simulate the behaviors of a traffic light.

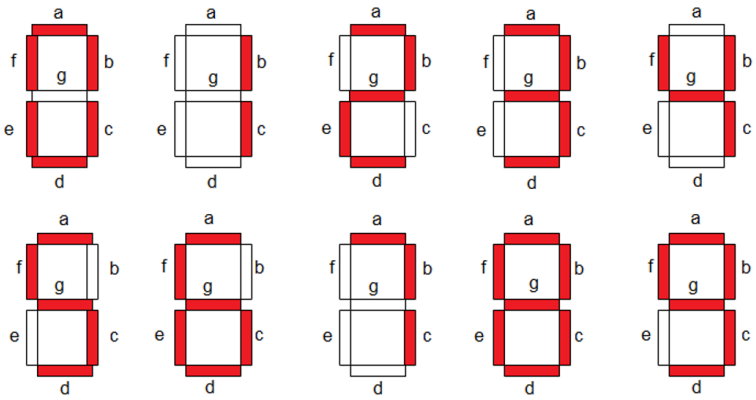


Hình 1.25: 4 way traffic light

4.4 Exercise 4

Add **only one 7 led segment** to the schematic in Exercise 3. This component can be found in Proteus by the keyword **7SEG-COM-ANODE**. For this device, the common pin should be connected to the power supply and other pins are supposed to connected to PB0 to PB6. Therefore, to turn-on a segment in this 7SEG, the STM32 pin should be in logic 0 (0V).

Implement a function named **display7SEG(int num)**. The input for this function is from 0 to 9 and the outputs are listed as following:

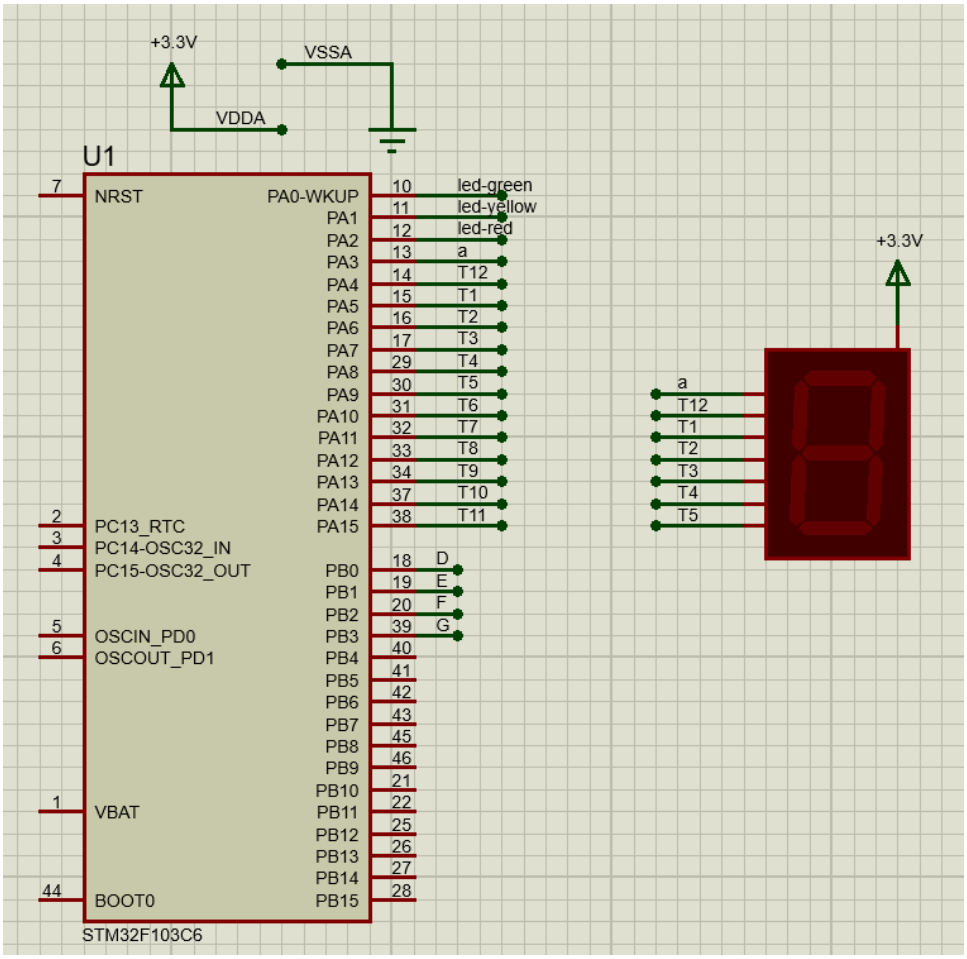


Hình 1.26: Display a number on 7 segment LED

This function is invoked in the while loop for testing as following:

```
int counter = 0;
while (1){
    if(counter >= 10) counter = 0;
    display7SEG(counter++);
    HAL_Delay(1000);

}
```

Report 1: Present the schematic.



Hình 1.27: 4 way traffic light

Report 2: Present the source code for display7SEG function.

```
#define a_Pin GPIO_PIN_3
#define b_Pin GPIO_PIN_4
#define c_Pin GPIO_PIN_5
#define d_Pin GPIO_PIN_6
#define e_Pin GPIO_PIN_7
#define f_Pin GPIO_PIN_8
#define g_Pin GPIO_PIN_9

#define a_GPIO_Port GPIOA
#define b_GPIO_Port GPIOA
#define c_GPIO_Port GPIOA
#define d_GPIO_Port GPIOA
#define e_GPIO_Port GPIOA
#define f_GPIO_Port GPIOA
#define g_GPIO_Port GPIOA
void display7SEG(int num){
  switch(num)
  {
  case 0:
    HAL_GPIO_WritePin(a_GPIO_Port, a_Pin, 0);
```

```
    HAL_GPIO_WritePin(b_GPIO_Port, b_Pin, 0);
    HAL_GPIO_WritePin(c_GPIO_Port, c_Pin, 0);
    HAL_GPIO_WritePin(d_GPIO_Port, d_Pin, 0);
    HAL_GPIO_WritePin(e_GPIO_Port, e_Pin, 0);
    HAL_GPIO_WritePin(f_GPIO_Port, f_Pin, 0);
    HAL_GPIO_WritePin(g_GPIO_Port, g_Pin, 1);
    break;
  case 1:
    HAL_GPIO_WritePin(a_GPIO_Port, a_Pin, 1);
    HAL_GPIO_WritePin(b_GPIO_Port, b_Pin, 0);
    HAL_GPIO_WritePin(c_GPIO_Port, c_Pin, 0);
    HAL_GPIO_WritePin(d_GPIO_Port, d_Pin, 1);
    HAL_GPIO_WritePin(e_GPIO_Port, e_Pin, 1);
    HAL_GPIO_WritePin(f_GPIO_Port, f_Pin, 1);
    HAL_GPIO_WritePin(g_GPIO_Port, g_Pin, 1);
    break;
  case 2:
    HAL_GPIO_WritePin(a_GPIO_Port, a_Pin, 0);
    HAL_GPIO_WritePin(b_GPIO_Port, b_Pin, 0);
    HAL_GPIO_WritePin(c_GPIO_Port, c_Pin, 1);
    HAL_GPIO_WritePin(d_GPIO_Port, d_Pin, 0);
    HAL_GPIO_WritePin(e_GPIO_Port, e_Pin, 0);
    HAL_GPIO_WritePin(f_GPIO_Port, f_Pin, 1);
    HAL_GPIO_WritePin(g_GPIO_Port, g_Pin, 0);
    break;
  case 3:
    HAL_GPIO_WritePin(a_GPIO_Port, a_Pin, 0);
    HAL_GPIO_WritePin(b_GPIO_Port, b_Pin, 0);
    HAL_GPIO_WritePin(c_GPIO_Port, c_Pin, 0);
    HAL_GPIO_WritePin(d_GPIO_Port, d_Pin, 0);
    HAL_GPIO_WritePin(e_GPIO_Port, e_Pin, 1);
    HAL_GPIO_WritePin(f_GPIO_Port, f_Pin, 1);
    HAL_GPIO_WritePin(g_GPIO_Port, g_Pin, 0);
    break;
  case 4:
    HAL_GPIO_WritePin(a_GPIO_Port, a_Pin, 1);
    HAL_GPIO_WritePin(b_GPIO_Port, b_Pin, 0);
    HAL_GPIO_WritePin(c_GPIO_Port, c_Pin, 0);
    HAL_GPIO_WritePin(d_GPIO_Port, d_Pin, 1);
    HAL_GPIO_WritePin(e_GPIO_Port, e_Pin, 1);
    HAL_GPIO_WritePin(f_GPIO_Port, f_Pin, 0);
    HAL_GPIO_WritePin(g_GPIO_Port, g_Pin, 0);
    break;
  case 5:
    HAL_GPIO_WritePin(a_GPIO_Port, a_Pin, 0);
    HAL_GPIO_WritePin(b_GPIO_Port, b_Pin, 1);
    HAL_GPIO_WritePin(c_GPIO_Port, c_Pin, 0);
    HAL_GPIO_WritePin(d_GPIO_Port, d_Pin, 0);
    HAL_GPIO_WritePin(e_GPIO_Port, e_Pin, 1);
```

```
    HAL_GPIO_WritePin(f_GPIO_Port, f_Pin, 0);
    HAL_GPIO_WritePin(g_GPIO_Port, g_Pin, 0);
    break;
  case 6:
    HAL_GPIO_WritePin(a_GPIO_Port, a_Pin, 0);
    HAL_GPIO_WritePin(b_GPIO_Port, b_Pin, 1);
    HAL_GPIO_WritePin(c_GPIO_Port, c_Pin, 0);
    HAL_GPIO_WritePin(d_GPIO_Port, d_Pin, 0);
    HAL_GPIO_WritePin(e_GPIO_Port, e_Pin, 0);
    HAL_GPIO_WritePin(f_GPIO_Port, f_Pin, 0);
    HAL_GPIO_WritePin(g_GPIO_Port, g_Pin, 0);
    break;
  case 7:
    HAL_GPIO_WritePin(a_GPIO_Port, a_Pin, 0);
    HAL_GPIO_WritePin(b_GPIO_Port, b_Pin, 0);
    HAL_GPIO_WritePin(c_GPIO_Port, c_Pin, 0);
    HAL_GPIO_WritePin(d_GPIO_Port, d_Pin, 1);
    HAL_GPIO_WritePin(e_GPIO_Port, e_Pin, 1);
    HAL_GPIO_WritePin(f_GPIO_Port, f_Pin, 1);
    HAL_GPIO_WritePin(g_GPIO_Port, g_Pin, 1);
    break;
  case 8:
    HAL_GPIO_WritePin(a_GPIO_Port, a_Pin, 0);
    HAL_GPIO_WritePin(b_GPIO_Port, b_Pin, 0);
    HAL_GPIO_WritePin(c_GPIO_Port, c_Pin, 0);
    HAL_GPIO_WritePin(d_GPIO_Port, d_Pin, 0);
    HAL_GPIO_WritePin(e_GPIO_Port, e_Pin, 0);
    HAL_GPIO_WritePin(f_GPIO_Port, f_Pin, 0);
    HAL_GPIO_WritePin(g_GPIO_Port, g_Pin, 0);
    break;
  case 9:
    HAL_GPIO_WritePin(a_GPIO_Port, a_Pin, 0);
    HAL_GPIO_WritePin(b_GPIO_Port, b_Pin, 0);
    HAL_GPIO_WritePin(c_GPIO_Port, c_Pin, 0);
    HAL_GPIO_WritePin(d_GPIO_Port, d_Pin, 0);
    HAL_GPIO_WritePin(e_GPIO_Port, e_Pin, 1);
    HAL_GPIO_WritePin(f_GPIO_Port, f_Pin, 0);
    HAL_GPIO_WritePin(g_GPIO_Port, g_Pin, 0);
    break;
  default: break;
  }
}
int counter = 0;
void ex4(){
  if(counter >= 10) counter = 0;
  display7SEG(counter++);
  HAL_Delay(1000);
}
while(1){
```

```
119     ex4();
120 }
```

4.5 Exercise 5

Integrate the 7SEG-LED to the 4 way traffic light. In this case, the 7SEG-LED is used to display countdown value.

In this exercise, only source code is required to present. The function display7SEG in previous exercise can be re-used.

```
1  #define LED_RED2_GPIO_Port       GPIOA
2  #define LED_RED2_Pin             GPIO_PIN_12
3  #define LED_YELLOW2_GPIO_Port    GPIOA
4  #define LED_YELLOW2_Pin          GPIO_PIN_11
5  #define LED_GREEN2_GPIO_Port     GPIOA
6  #define LED_GREEN2_Pin           GPIO_PIN_10
7
8  #define A_Pin GPIO_PIN_13
9  #define B_Pin GPIO_PIN_14
10 #define C_Pin GPIO_PIN_15
11 #define D_Pin GPIO_PIN_0
12 #define E_Pin GPIO_PIN_1
13 #define F_Pin GPIO_PIN_2
14 #define G_Pin GPIO_PIN_3
15
16 #define A_GPIO_Port GPIOA
17 #define B_GPIO_Port GPIOA
18 #define C_GPIO_Port GPIOA
19 #define D_GPIO_Port GPIOB
20 #define E_GPIO_Port GPIOB
21 #define F_GPIO_Port GPIOB
22 #define G_GPIO_Port GPIOB
23
24 void display7SEG_2(int num){
25     switch(num){
26     case 0:
27       HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, 0);
28       HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, 0);
29       HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, 0);
30       HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, 0);
31       HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, 0);
32       HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, 0);
33       HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, 1);
34       break;
35     case 1:
36       HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, 1);
37       HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, 0);
```

```
38        HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, 0);
39        HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, 1);
40        HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, 1);
41        HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, 1);
42        HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, 1);
43        break;
44      case 2:
45        HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, 0);
46        HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, 0);
47        HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, 1);
48        HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, 0);
49        HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, 0);
50        HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, 1);
51        HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, 0);
52        break;
53      case 3:
54        HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, 0);
55        HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, 0);
56        HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, 0);
57        HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, 0);
58        HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, 1);
59        HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, 1);
60        HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, 0);
61        break;
62      case 4:
63        HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, 1);
64        HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, 0);
65        HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, 0);
66        HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, 1);
67        HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, 1);
68        HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, 0);
69        HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, 0);
70        break;
71      case 5:
72        HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, 0);
73        HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, 1);
74        HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, 0);
75        HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, 0);
76        HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, 1);
77        HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, 0);
78        HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, 0);
79        break;
80      case 6:
81        HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, 0);
82        HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, 1);
83        HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, 0);
84        HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, 0);
85        HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, 0);
86        HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, 0);
```

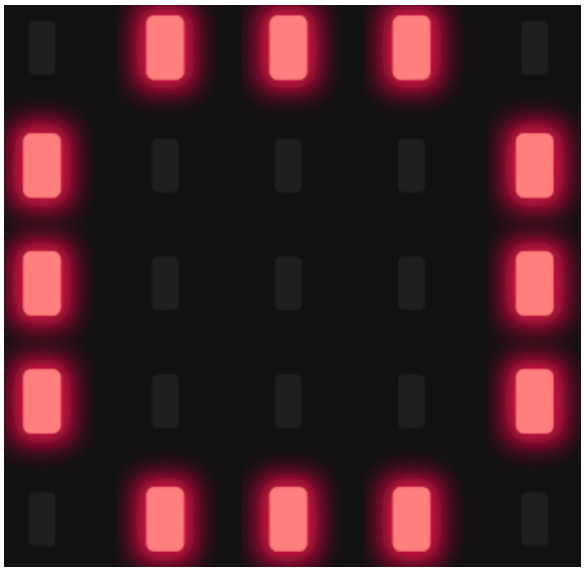
```
87        HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, 0);
88        break;
89      case 7:
90        HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, 0);
91        HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, 0);
92        HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, 0);
93        HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, 1);
94        HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, 1);
95        HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, 1);
96        HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, 1);
97        break;
98      case 8:
99        HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, 0);
100       HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, 0);
101       HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, 0);
102       HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, 0);
103       HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, 0);
104       HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, 0);
105       HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, 0);
106       break;
107     case 9:
108       HAL_GPIO_WritePin(A_GPIO_Port, A_Pin, 0);
109       HAL_GPIO_WritePin(B_GPIO_Port, B_Pin, 0);
110       HAL_GPIO_WritePin(C_GPIO_Port, C_Pin, 0);
111       HAL_GPIO_WritePin(D_GPIO_Port, D_Pin, 0);
112       HAL_GPIO_WritePin(E_GPIO_Port, E_Pin, 1);
113       HAL_GPIO_WritePin(F_GPIO_Port, F_Pin, 0);
114       HAL_GPIO_WritePin(G_GPIO_Port, G_Pin, 0);
115       break;
116     default: break;
117     }
118   }
119 int Count = 9;
120 void ex5()
121 {
122   if(Count < 2){
123     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, 1);
124     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
      0);
125     HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
    0);
126     display7SEG(Count);
127
128     HAL_GPIO_WritePin(LED_RED2_GPIO_Port, LED_RED2_Pin, 0);
129     HAL_GPIO_WritePin(LED_YELLOW2_GPIO_Port,
    LED_YELLOW2_Pin, 1);
130     HAL_GPIO_WritePin(LED_GREEN2_GPIO_Port, LED_GREEN2_Pin,
      0);
131     display7SEG_2(Count);
```

```
132   }
133   else if(Count < 5){
134     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, 1);
135     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
      0);
136     HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
     0);
137     display7SEG(Count);
138
139     HAL_GPIO_WritePin(LED_RED2_GPIO_Port, LED_RED2_Pin, 0);
140     HAL_GPIO_WritePin(LED_YELLOW2_GPIO_Port,
     LED_YELLOW2_Pin, 0);
141     HAL_GPIO_WritePin(LED_GREEN2_GPIO_Port, LED_GREEN2_Pin,
      1);
142     display7SEG_2(Count-2);
143   }
144   else if(Count < 7){
145     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, 0);
146     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
      1);
147     HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
     0);
148     display7SEG(Count-5);
149
150     HAL_GPIO_WritePin(LED_RED2_GPIO_Port, LED_RED2_Pin, 1);
151     HAL_GPIO_WritePin(LED_YELLOW2_GPIO_Port,
     LED_YELLOW2_Pin, 0);
152     HAL_GPIO_WritePin(LED_GREEN2_GPIO_Port, LED_GREEN2_Pin,
      0);
153     display7SEG_2(Count-5);
154   }
155   else{
156     HAL_GPIO_WritePin(LED_RED_GPIO_Port, LED_RED_Pin, 0);
157     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port, LED_YELLOW_Pin,
      0);
158     HAL_GPIO_WritePin(LED_GREEN_GPIO_Port, LED_GREEN_Pin,
     1);
159     display7SEG(Count-7);
160
161     HAL_GPIO_WritePin(LED_RED2_GPIO_Port, LED_RED2_Pin, 1);
162     HAL_GPIO_WritePin(LED_YELLOW2_GPIO_Port,
     LED_YELLOW2_Pin, 0);
163     HAL_GPIO_WritePin(LED_GREEN2_GPIO_Port, LED_GREEN2_Pin,
      0);
164     display7SEG_2(Count-5);
165   }
166   Count--;
167   if(Count < 0){
168     Count = 9;
```

```
169   }
170     HAL_Delay(1000);
171 }
172 while(1){
173     ex5();
174 }
```

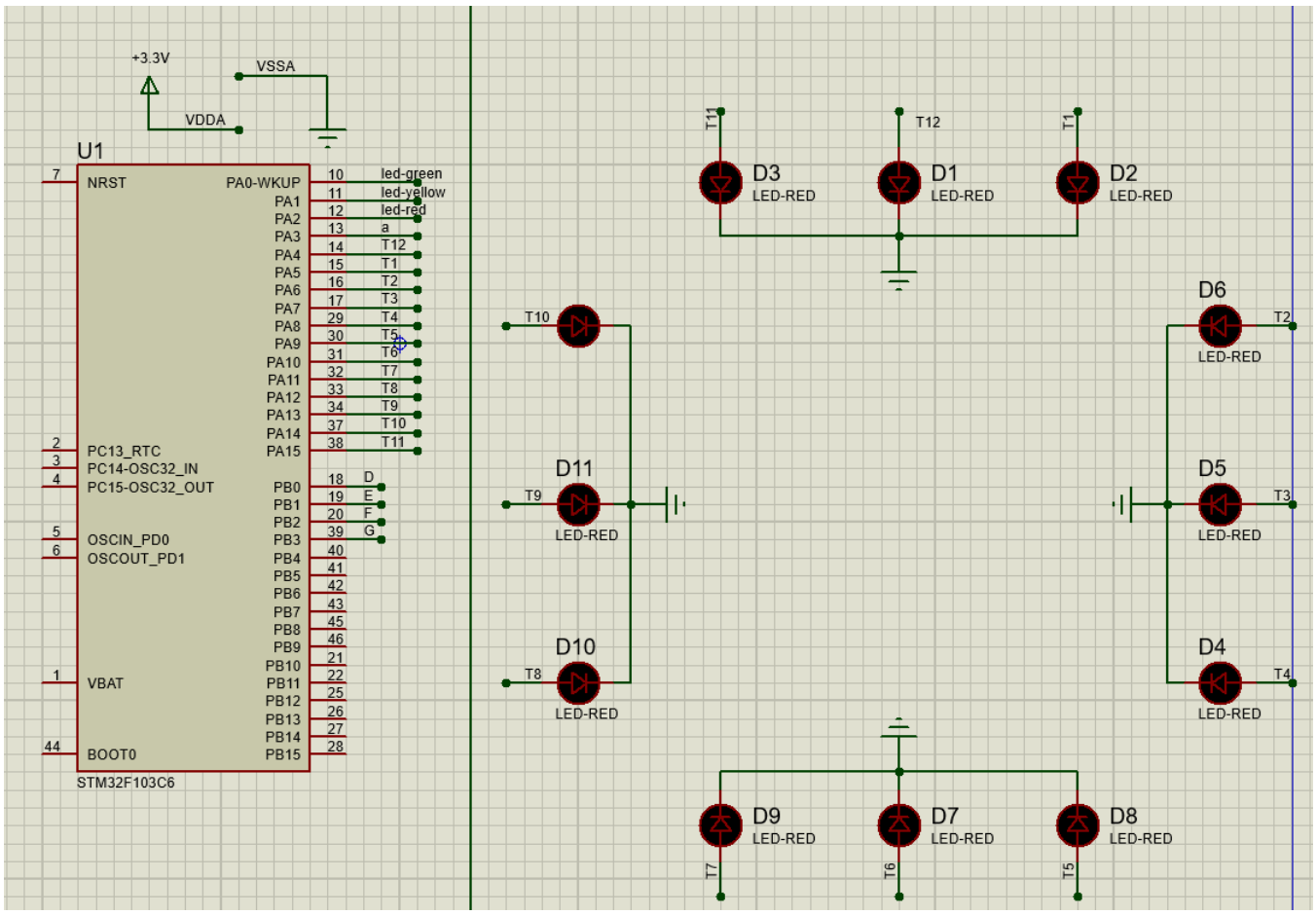
4.6 Exercise 6

In this exercise, a new Proteus schematic is designed to simulate an analog clock, with 12 different number. The connections for 12 LEDs are supposed from PA4 to PA15 of the STM32. The arrangement of 12 LEDs is depicted as follows.



Hình 1.28: 12 LEDs for an analog clock

Report 1: Present the schematic.



Report 2: Implement a simple program to test the connection of every single LED. This testing program should turn every LED in a sequence.

```
void ex6(){
  for(int i = start; i <= end; i *= 2){
    HAL_GPIO_WritePin(GPIOA, i, 1);
    HAL_Delay(1000);
    HAL_GPIO_WritePin(GPIOA, i, 0);
  }
}
```

4.7 Exercise 7

Implement a function named **clearAllClock()** to turn off all 12 LEDs. Present the source code of this function.

```
#define start GPIO_PIN_4
#define end   GPIO_PIN_15
void clearAllClock (){
  for(int i = start; i <= end; i *= 2){
    HAL_GPIO_WritePin(GPIOA, i, 0);
  }
}
```

Program 1.3: Function Implementation

4.8 Exercise 8

Implement a function named **setNumberOnClock(int num)**. The input for this function is from **0 to 11** and an appropriate LED is turn on. Present the source code of this function.

```
void setNumberOnClock(int num){
    switch (num){
    case 0:
      HAL_GPIO_WritePin(T12_GPIO_Port, T12_Pin, 1);
      break;
    case 1:
      HAL_GPIO_WritePin(T1_GPIO_Port, T1_Pin, 1);
      break;
    case 2:
      HAL_GPIO_WritePin(T2_GPIO_Port, T2_Pin, 1);
      break;
    case 3:
      HAL_GPIO_WritePin(T3_GPIO_Port, T3_Pin, 1);
      break;
    case 4:
      HAL_GPIO_WritePin(T4_GPIO_Port, T4_Pin, 1);
      break;
    case 5:
      HAL_GPIO_WritePin(T5_GPIO_Port, T5_Pin, 1);
      break;
    case 6:
      HAL_GPIO_WritePin(T6_GPIO_Port, T6_Pin, 1);
      break;
    case 7:
      HAL_GPIO_WritePin(T7_GPIO_Port, T7_Pin, 1);
      break;
    case 8:
      HAL_GPIO_WritePin(T8_GPIO_Port, T8_Pin, 1);
      break;
    case 9:
      HAL_GPIO_WritePin(T9_GPIO_Port, T9_Pin, 1);
      break;
    case 10:
      HAL_GPIO_WritePin(T10_GPIO_Port, T10_Pin, 1);
      break;
    case 11:
      HAL_GPIO_WritePin(T11_GPIO_Port, T11_Pin, 1);
      break;
    default: break;
    }
  }
```

Program 1.4: Function Implementation

4.9 Exercise 9

Implement a function named **clearNumberOnClock(int num)**. The input for this function is from **0 to 11** and an appropriate LED is turn off.

```
void clearNumberOnClock(int num){
    switch (num){
        case 0:
            HAL_GPIO_WritePin(T12_GPIO_Port, T12_Pin, 0);
    break;
  case 1:
    HAL_GPIO_WritePin(T1_GPIO_Port, T1_Pin, 0);
    break;
  case 2:
    HAL_GPIO_WritePin(T2_GPIO_Port, T2_Pin, 0);
    break;
  case 3:
    HAL_GPIO_WritePin(T3_GPIO_Port, T3_Pin, 0);
    break;
  case 4:
    HAL_GPIO_WritePin(T4_GPIO_Port, T4_Pin, 0);
    break;
  case 5:
    HAL_GPIO_WritePin(T5_GPIO_Port, T5_Pin, 0);
    break;
  case 6:
    HAL_GPIO_WritePin(T6_GPIO_Port, T6_Pin, 0);
    break;
  case 7:
    HAL_GPIO_WritePin(T7_GPIO_Port, T7_Pin, 0);
    break;
  case 8:
    HAL_GPIO_WritePin(T8_GPIO_Port, T8_Pin, 0);
    break;
  case 9:
    HAL_GPIO_WritePin(T9_GPIO_Port, T9_Pin, 0);
    break;
  case 10:
     HAL_GPIO_WritePin(T10_GPIO_Port, T10_Pin, 0);
    break;
  case 11:
    HAL_GPIO_WritePin(T11_GPIO_Port, T11_Pin, 0);
    break;
  default: break;
    }
}
```

Program 1.5: Function Implementation

4.10 Exercise 10

Integrate the whole system and use 12 LEDs to display a clock. At a given time, there are only 3 LEDs are turn on for hour, minute and second information.

```
int sec = 0, min = 0, hour = 0;
void ex10_init(){
  clearAllClock();
  setNumberOnClock(0);
}
void ex10(){
  HAL_Delay(100);
  clearNumberOnClock(sec/5);
  clearNumberOnClock(min/5);
  clearNumberOnClock(hour);

  sec++;
  if(sec >= 60){
    sec = 0;
    min++;
  }

  if(min >= 60){
    min = 0;
    hour++;
  }

  if(hour >= 12){
    hour = 0;
  }
  setNumberOnClock(sec/5);
  setNumberOnClock(min/5);
  setNumberOnClock(hour);
}
ex10_init();
while (1)
{
    ex10();
}
```

Program 1.6: Function Implementation

5 Source code and simulation files

All source code and simulation files are pushed to github.
Click here to access files